Practical No: 1(Data Pre-processing and Exploration) (1abc)
a. Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.
b. Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and
identify potential features and target variables
c. Create or Explore datasets to use all pre-processing routines like label encoding, scaling.
Practical No: 3 (3a)
Simple Linear Regression
A. Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE
PRACTICAL 4 (4a)
Aim: Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve.
PRACTICAL 5(4d)
Aim: Implement a Support Vector Machine for any relevant dataset.
PRACTICAL 6 (6a)
Aim: To implement the Naive Bayes classification algorithm using Python to predict whether a person should play based on weather conditions and temperature, and evaluate the model's accuracy.
PRACTICAL 7(4b)
Aim: Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a .CSV file and build the model to classify a test sample. Print both correct and wrong predictions.

Cmd
1. py -m pip install --upgrade pip
2. py -m pip install jupyterlab notebook ipykernel numpy pandas matplotlib seaborn scikit-learn scipy
3. cd /d "File location folder name"
4. py -m notebook
5. py -m pip install numpy pandas matplotlib seaborn scikit-learn scipy

Practical No: 1(Data Pre-processing and Exploration) (1abc)
a. Load a CSV dataset. Handle missing values, inconsistent formatting, and outliers.

```
# Install: python -m pip install pandas numpy scikit-learn
import pandas as pd
import numpy as np
from sklearn.impute import SimpleImputer

df = pd.DataFrame({
    "Age": [22, 25, np.nan, 120, 29, 31],
    "Salary": [25000, 32000, 28000, 50000, np.nan, 42000],
    "City": ["mumbai", "PUNE", "Mumbai ", "Navi Mumbai", None, "pune"]
})
print("Original data:\n", df)
df["City"] = df["City"].str.strip().str.title().fillna("Unknown")
num_cols = ["Age", "Salary"]
df[num_cols] = SimpleImputer(strategy="median").fit_transform(df[num_cols])
# Cap numeric outliers with IQR bounds
for col in num_cols:
    q1, q3 = df[col].quantile([0.25, 0.75])
    iqr = q3 - q1
```

```
    df[col] = df[col].clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
print("\nCleaned data:\n", df)
print("\nMissing values:\n", df.isnull().sum())
```

b. Load a dataset, calculate descriptive summary statistics, create visualizations using different graphs, and identify potential features and target variables

```
# Install: python -m pip install pandas numpy matplotlib seaborn
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = sns.load_dataset("penguins").dropna()
print(df.head())
print("\nShape:", df.shape)
print("\nSummary statistics:\n", df.describe(include="all"))
print("\nMissing values:\n", df.isnull().sum())
numeric = df.select_dtypes(include="number").columns
for col in numeric:
    sns.histplot(data=df, x=col, kde=True)
    plt.title(f"Distribution of {col}")
    plt.tight_layout()
    plt.show()
sns.countplot(data=df, x="species")
plt.title("Penguin species counts"); plt.tight_layout(); plt.show()
sns.scatterplot(data=df, x="bill_length_mm", y="bill_depth_mm", hue="species")
plt.title("Bill length vs bill depth"); plt.tight_layout(); plt.show()
sns.heatmap(df[numeric].corr(), annot=True, cmap="coolwarm")
plt.title("Correlation heatmap"); plt.tight_layout(); plt.show()
print("\nPotential target: species")
print("Potential features: bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g")
```

c. Create or Explore datasets to use all pre-processing routines like label encoding, scaling

```
# Install: python -m pip install pandas scikit-learn
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler, Binarizer

df = pd.DataFrame({
    "City": ["Mumbai", "Pune", "Nashik", "Mumbai"],
    "Age": [21, 25, 30, 23],
    "Score": [45, 70, 85, 60]
})
print("Original:\n", df)
le = LabelEncoder()
df["City_Label"] = le.fit_transform(df["City"])
df["Age_Standardized"] = StandardScaler().fit_transform(df[["Age"]])
df["Score_MinMax"] = MinMaxScaler().fit_transform(df[["Score"]])
```

```
df["Score_Binary"] = Binarizer(threshold=60).fit_transform(df[["Score"]]).astype(int)
print("\nTransformed:\n", df)
print("\nCity label mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
```

Practical No: 3 (3a)

Simple Linear Regression

A. Fit a linear regression model on a dataset. Interpret coefficients, make predictions, and evaluate performance using metrics like R-squared and MSE

```
# Install: python -m pip install numpy matplotlib scikit-learn
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([15, 18, 21, 25, 28, 31, 35, 38, 41, 45])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
model = LinearRegression().fit(X_train, y_train)
pred = model.predict(X_test)
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
print("Predictions:", pred)
print("MSE:", mean_squared_error(y_test, pred))
print("R-squared:", r2_score(y_test, pred))
plt.scatter(X, y, label="Data")
plt.plot(X, model.predict(X), label="Regression line")
plt.xlabel("Study hours"); plt.ylabel("Score"); plt.legend(); plt.show()
```

PRACTICAL 4 (4a)

Aim: Perform binary classification using logistic regression. Calculate accuracy, precision, recall, and understand the ROC curve

```
# Install: python -m pip install scikit-learn matplotlib
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             classification_report, RocCurveDisplay)

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=3000))
```

```
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Precision:", round(precision_score(y_test, pred), 4))
print("Recall:", round(recall_score(y_test, pred), 4))
print(classification_report(y_test, pred))
RocCurveDisplay.from_estimator(model, X_test, y_test)
plt.title("ROC Curve"); plt.show()
```

PRACTICAL 5(4d)

Aim: Implement a Support Vector Machine for any relevant dataset.

```
# Install: python -m pip install scikit-learn
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report

X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = make_pipeline(StandardScaler(), SVC(kernel="rbf", C=1.0, gamma="scale"))
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print(classification_report(y_test, pred))
```

PRACTICAL 6 (6a)

Aim: To implement the Naive Bayes classification algorithm using Python to predict whether a person should play based on weather conditions and temperature, and evaluate the model's accuracy.

```
# Install: python -m pip install scikit-learn
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42, stratify=data.target
)
model = GaussianNB()
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Prior class probabilities:", model.class_prior_)
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("Confusion matrix:\n", confusion_matrix(y_test, pred))
```

```
print(classification_report(y_test, pred, target_names=data.target_names))
sample = X_test[:1]
print("Sample features:", sample[0])
print("Posterior probabilities:", model.predict_proba(sample)[0])
print("Predicted class:", data.target_names[model.predict(sample)[0]])
```

PRACTICAL 7(4b)

Aim: Implement and demonstrate k-nearest Neighbor algorithm. Read the training data from a .CSV file and build the model to classify a test sample. Print both correct and wrong predictions.

```
# Install: python -m pip install pandas scikit-learn
import pandas as pd
from pathlib import Path
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

path = Path("knn_data.csv")
if not path.exists():
    pd.DataFrame({
        "Height": [150, 155, 160, 162, 165, 170, 172, 175, 180, 185, 158, 178],
        "Weight": [48, 50, 52, 54, 57, 60, 62, 65, 72, 78, 51, 68],
        "Class": ["A", "A", "A", "A", "A", "B", "B", "B", "B", "B", "A", "B"]
    }).to_csv(path, index=False)
    print("Created sample CSV:", path.resolve())
df = pd.read_csv(path)
X, y = df.drop(columns="Class"), df["Class"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=3))
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", round(accuracy_score(y_test, pred), 4))
print("\nCorrect predictions:")
for actual, guessed in zip(y_test.to_numpy(), pred):
    if actual == guessed: print(f"Actual={actual}, Predicted={guessed}")
print("\nIncorrect predictions:")
for actual, guessed in zip(y_test.to_numpy(), pred):
    if actual != guessed: print(f"Actual={actual}, Predicted={guessed}")
```